Black-Box and White-Box Testing, Test Design Techniques, and Regression Testing

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Differentiate black-box and white-box testing by their focus and evidence.

• Design tests using equivalence partitioning and boundary value analysis.

• Recognize decision-table and state-transition testing.

• Understand statement and branch coverage at an introductory level.

• Explain regression testing and select a sensible regression suite.

Running case study: College Event Management System (CEMS), where students register for events subject to registration status, capacity, and duplicate-registration rules.

1. Two Ways to Look for Defects

Black-box testing

Black-box testing checks externally observable behavior against requirements without relying on knowledge of internal code. The tester supplies inputs or performs actions, then compares actual results with expected results.

Example: Submit a registration when an event is full. The expected result is a clear rejection and no new registration record. The test is based on the requirement, not on how the code implements the capacity check.

White-box testing

White-box testing uses knowledge of internal program structure to design or evaluate tests. It may examine statements, branches, conditions, paths, and data flow.

Example: Inspect the registration method and create tests that execute both the “capacity available” and “event full” branches.

Aspect	Black-box	White-box
Basis	Requirements and observable behavior	Code structure and logic
Typical question	Does the feature behave correctly for these inputs?	Have important statements and branches been exercised?
Who can perform it?	Testers, developers, users, depending on context	Usually developers or testers with code access
Finds well	Missing behavior, incorrect outputs, interface and requirement mismatches	Unexecuted logic, branch errors, certain logic and control-flow defects
Limitation	May miss hidden internal paths	May miss missing requirements or behavior not represented in the code

These approaches complement each other. Neither one alone guarantees that the software is defect-free.

2. Equivalence Partitioning

Equivalence partitioning divides input values into groups expected to be handled similarly. Instead of testing every possible value, select representative values from valid and invalid groups.

Example requirement: A student can register only when the event is open and has available capacity. Suppose a registration form accepts an attendee count from 1 to 4.

Partition	Representative input	Expected behavior
Valid count: 1–4	2	Accept, if other rules pass
Invalid count: below minimum	0	Reject with validation message
Invalid count: above maximum	5	Reject with validation message
Invalid format	“two” or blank	Reject if numeric count is required

The representative input should be chosen because it belongs to the partition, not because it seems likely to reveal a defect. Additional tests may be needed when different values trigger different rules.

3. Boundary Value Analysis

Defects often occur near limits. Boundary value analysis selects values at, just below, and just above a boundary.

Example: attendee count must be between 1 and 4.

Test value	Reason	Expected result
0	Just below minimum	Reject
1	Minimum boundary	Accept, if other rules pass
2	Typical valid value	Accept, if other rules pass
4	Maximum boundary	Accept, if other rules pass
5	Just above maximum	Reject

> For a registration-capacity rule with capacity 100, test counts or remaining seats around 99, 100, and 101 as appropriate to the exact requirement. Be precise about whether the input represents registered students, available seats, or requested attendees.

4. Decision-Table Testing

Decision tables help when behavior depends on combinations of conditions. List the conditions, possible values, and expected action for each meaningful rule combination.

Registration open?	Capacity available?	Already registered?	Expected action
Yes	Yes	No	Allow registration
No	Yes	No	Reject: registration closed
Yes	No	No	Reject: event full
Yes	Yes	Yes	Prevent duplicate registration
No	No	No	Reject; show the most useful applicable message

The final row illustrates that several conditions may fail simultaneously. Product owners should define the desired priority of messages and actions so that implementation and tests remain consistent.

5. State-Transition Testing

State-transition testing checks whether the system responds correctly when an entity moves between states.

Example event states: Draft → Published → Registration Open → Registration Closed → Completed. Cancellation may be allowed from some states but not others.

Current state	Action / event	Expected next state
Draft	Organizer publishes event	Published
Published	Registration period opens	Registration Open
Registration Open	Deadline passes	Registration Closed

Current state	Action / event	Expected next state
Registration Closed	Event date passes and event finishes	Completed
Completed	Student attempts new registration	No transition; registration rejected

Test both permitted and forbidden transitions. A system that correctly handles normal transitions but allows an invalid transition can still violate its business rules.

6. White-Box Coverage: Statements and Branches

Statement coverage

Statement coverage measures the proportion of executable statements exercised by a test suite. If 8 of 10 executable statements are exercised, statement coverage is 80%.

Branch coverage

Branch coverage measures whether each outcome of a decision has been exercised. For an if/else decision, tests should exercise both the true and false outcomes.

Illustrative logic:

```
if registrationOpen AND hasCapacity:
createRegistration()
else:
rejectRegistration()
```

A test where registration is open and capacity remains may execute the success branch. Another test where the event is closed or full should execute the rejection branch. Branch coverage does not guarantee that every possible combination of conditions or every execution path has been tested.

7. Regression Testing

Regression testing checks that a change has not unintentionally broken existing behavior. Changes include bug fixes, new features, refactoring, configuration updates, or dependency upgrades.

CEMS change: The team updates the registration service to support waitlists. Regression checks should include ordinary registration, full-event rejection, duplicate prevention, organizer attendee lists, cancellation, and relevant notifications.

Selecting a regression suite

• Prioritize tests around the changed component and its dependencies.

• Include critical business workflows and high-impact failure cases.

• Reuse stable automated tests where appropriate.

• Include integration tests where components communicate.

• Run broader regression checks before major releases.

• Keep expected results and test data maintained as requirements evolve.

A smoke test is a quick check that critical functionality is working after a build or deployment. A regression suite is broader and specifically checks for unintended effects of changes; a smoke test may be part of it but is not the same thing.

8. Writing a Strong Test Case

Field	What to record
Test ID and title	Unique reference and concise purpose
Requirement / risk	What rule or risk is being checked
Preconditions	Required account, event status, capacity, and data
Steps and inputs	Reproducible actions and values

Field	What to record
Expected result	Specific observable behavior
Actual result	What happened during execution
Status and evidence	Pass/fail, environment, screenshot or logs

Weak expected result: “It should work.” Strong expected result: “The system rejects registration, displays ‘Event is full’, and creates no new registration record.”

9. Practical Exercise

Create a test suite for CEMS registration using all four techniques.

1. Equivalence partitioning: define valid and invalid attendee-count groups.

2. Boundary value analysis: test just below, at, and just above the permitted count limits.

3. Decision table: cover open/closed, capacity available/full, and duplicate/not-duplicate conditions.

4. State-transition testing: test at least two valid transitions and two invalid transitions.

5. Regression planning: list five existing tests that should be rerun after adding a waitlist feature.

6. White-box planning: identify the true and false branches in a simple registration condition.

Deliverable: a test design sheet with test IDs, inputs, preconditions, expected results, and a short rationale for each selected test.

10. Review Questions with Suggested Answers

1. What distinguishes black-box from white-box testing?

Suggested answer: Black-box testing focuses on observable behavior and requirements; white-box testing uses internal code structure.

2. What is equivalence partitioning?

Suggested answer: Grouping inputs expected to behave similarly and selecting representative values from each group.

3. Why test boundary values?

Suggested answer: Errors commonly occur at the edges of valid ranges or around limits.

4. What is a decision table useful for?

Suggested answer: Testing combinations of conditions and the actions required for each combination.

5. What does state-transition testing check?

Suggested answer: Valid and invalid transitions between states in response to events.

6. Define statement coverage.

Suggested answer: The proportion of executable statements exercised by a test suite.

7. Does branch coverage prove all paths are tested?

Suggested answer: No. It shows branch outcomes were exercised, not that every path or condition combination was covered.

8. What is regression testing?

Suggested answer: Checking that changes have not unintentionally broken existing functionality.

9. How does a smoke test differ from regression testing?

Suggested answer: A smoke test is a quick critical-function check; regression testing is a broader check for unintended effects of changes.

10. What makes an expected result useful?

Suggested answer: It is specific, observable, and tied to the requirement or business rule.

11. Glossary

Term	Meaning
Black-box testing	Testing behavior without relying on internal code knowledge
White-box testing	Testing informed by internal code structure
Equivalence partitioning	Grouping inputs into classes expected to behave similarly
Boundary value analysis	Testing values at and around input limits
Decision table	A table mapping condition combinations to expected actions
State-transition testing	Testing state changes and allowed or forbidden transitions
Statement coverage	Measure of executable statements exercised
Branch coverage	Measure of decision outcomes exercised
Regression testing	Checking existing behavior after a change
Smoke testing	Quick check of critical functionality after a build or deployment

12. Homework

1. Design test cases for a student marks-entry form that accepts marks from 0 to 100.

2. Create a decision table for login with valid/invalid username, valid/invalid password, and locked/unlocked account.

3. Draw a state diagram for an online order and write tests for two invalid transitions.

4. A code branch has a true and false outcome. Propose tests that exercise both outcomes and explain what coverage still does not guarantee.

5. After a bug fix to event cancellation, propose a focused regression suite and justify each test.

Next chapter: Software Quality Assurance, Software Metrics, and Software Maintenance.